

CSCE 637 COMPLEXITY THEORY · MODULE 03 · LECTURE NOTES

NP, and Verification

The class, and the question it poses.

MODULE OUTCOMES

Define NP by certificates and by non-determinism, and prove them equivalent.

Explain why the certificate definition is the useful one.

Place problems in NP by exhibiting certificates.

State P versus NP precisely and explain what it asks.

Explain what a resolution either way would mean.

1 The two definitions

NP IS 'VERIFIABLE WITH A SHORT CERTIFICATE', AND THAT IS THE DEFINITION TO USE

Certificate definition: a language L is in NP if there is a polynomial-time verifier V and a polynomial p such that x is in L exactly when there exists a certificate c with $|c|$ at most $p(|x|)$ for which $V(x, c)$ accepts.

Non-determinism definition: L is in NP if some non-deterministic Turing machine decides it in polynomial time — which is where the N comes from, and note it is *non-deterministic* rather than *non-polynomial*, a confusion worth heading off.

They are equivalent, and the proof is immediate in both directions — the certificate is precisely the sequence of non-deterministic choices the machine would make, and the verifier is the machine following them deterministically.

And the certificate definition is the one to use in practice, because it makes membership proofs concrete: to show a problem is in NP, state what the certificate is and how it is checked in polynomial time. **Two sentences, every time** (§1's table) — whereas the non-deterministic formulation requires describing a machine.

Problem	Certificate	Checked in polynomial time by
SAT	A satisfying truth assignment.	Evaluating the formula under it.
Hamiltonian cycle	The cycle, as a vertex sequence.	Checking each consecutive pair is an edge and that every vertex appears exactly once.
Compositeness	A non-trivial factor.	One division. Note the contrast with primality, whose certificate is far less obvious — which is why PRIMES in co-NP, then NP, then P took decades.

Problem	Certificate	Checked in polynomial time by
Subset sum	The subset achieving the target.	Adding it up.
Graph isomorphism	The vertex mapping between the two graphs.	Checking that edges correspond under the mapping.
Sudoku solvability	The completed grid.	Checking every row, column, and box.

If you can name the certificate in a sentence, the problem is in NP — which makes the membership half of an NP-completeness proof routine, and it is the half people omit. A hardness proof alone establishes NP-hardness, not NP-completeness, and the distinction matters because NP-hardness is compatible with the problem being far harder than NP (Module 05).

2 The question

P VERSUS NP ASKS WHETHER VERIFICATION IS GENUINELY EASIER THAN SEARCH

P is 'solvable quickly'. **NP** is 'checkable quickly, given the right answer'. **P** is contained in **NP** trivially — if you can solve a problem you can verify a claimed solution by ignoring the certificate and solving it yourself.

So **P = NP** would mean that whenever a solution can be recognised, it can be found — that checking and finding are the same difficulty, up to a polynomial.

Which is why the question matters well outside the field. Mathematics is the search for proofs, and a proof is checkable (CSCE 627 Module 10 §1); engineering design is the search for an artefact meeting criteria that can be checked; so is drug design, circuit layout, and scheduling. **P = NP** would make all of those mechanical.

And the question has been open since Cook's 1971 paper (and Levin's independent 1973 work), with a widely shared belief that **P ≠ NP** and no proof in either direction — which Modules 07 and 09 explain rather than excuse, since the field has theorems about why its techniques are insufficient.

If P = NP	If P ≠ NP
Public-key cryptography as deployed breaks — the problems it rests on are in NP.	Cryptography's assumptions remain plausible — though it needs considerably more than P ≠ NP , since it needs average-case hardness and one-way functions (CSCE 711).
Optimisation becomes tractable — scheduling, routing, layout, protein design, all of it.	The approximation and parameterised work of Modules 11 and 12 is the correct response, which is what the field has been doing.
Mathematical proof search becomes automatic for statements with short proofs, which is most of them.	Proof remains a creative act, in a provable sense — which is a striking thing for a theorem to establish.

If $P = NP$	If $P \neq NP$
And the constant and exponent would matter enormously — an n^{20} algorithm for SAT would be a mathematical earthquake and a practical irrelevance.	And we still would not know <i>how hard</i> — even quadratic lower bounds for SAT are not proved, so $P \neq NP$ would leave most quantitative questions open.

Note the asymmetry: $P = NP$ would be transformative, and $P \neq NP$ would change almost nothing operationally, because everyone already assumes it and has built accordingly. **Which is part of why the proof effort is intellectual rather than practical**, and is worth saying when the question's importance is being justified by its applications.

3 The evidence

- **Fifty years of serious effort on thousands of NP-complete problems has produced no polynomial algorithm for any of them. The best single argument**, and it is a straightforwardly empirical one.
- **And a polynomial algorithm for any one of them would give one for all of them** (Module 04 §1), **so the failures are jointly informative rather than separately** — thousands of independent research programmes all failing at what is formally one task.
- **The hierarchy theorems** (Module 07) **establish that more time genuinely does buy more computational power**, so the general shape of the belief is at least consistent with what is proved — the alternative would be strange.
- **Circuit lower bounds for restricted models** (Module 09) **are real, non-trivial, and in the expected direction** — monotone circuits, bounded-depth circuits, and certain algebraic models all have proved exponential lower bounds for NP-complete problems.
- **And the honest counterweight: all of this is circumstantial, and the field has proved theorems stating that its own principal techniques are inadequate to settle the question** (Module 07 §4's relativisation and Module 09 §3's natural proofs). **A field that can prove its own methods insufficient has reason for humility about its beliefs** — and stating that is more useful than reciting the consensus.

4 NP's structure

IF $P \neq NP$, THERE ARE PROBLEMS STRICTLY BETWEEN

Ladner's theorem: if $P \neq NP$, then there exist problems in NP that are neither in P nor NP-complete. NP-intermediate problems exist, conditionally on the central conjecture.

The proof is a diagonalisation (CSCE 627 Module 05 §3) **constructing a language that is deliberately made too hard to be in P and too easy to be complete**, by alternating between the two behaviours on sparse intervals — **and the constructed problem is entirely artificial**, which is the usual character of a diagonalisation's output.

The natural candidates are graph isomorphism and integer factoring (Module 02 §2), both of which have resisted both classifications for decades despite substantial attention.

So NP is not a two-tier structure of easy and complete. Which matters when you meet a problem that resists both a polynomial algorithm and a hardness proof: that may be the correct answer rather than a failure on your part, and knowing the middle exists prevents a certain amount of wasted effort in both directions.

- **Use the certificate definition. Membership in NP is two sentences: the witness, and how it is checked** (§1).
- **P versus NP is 'is finding as easy as checking'**, which is a natural question and **explains why it matters beyond complexity theory** (§2).

- **The belief that $P \neq NP$ is empirically grounded and not proved, and the field knows, as a theorem, why it has not been able to prove it.**
- **NP has a middle**, by Ladner, so **'neither' is a real classification** rather than an admission of defeat.
- **And $P = NP$ would be transformative while $P \neq NP$ would change almost nothing** — the practical world has already assumed it for fifty years. **The certificate habit is the single most useful thing in this module**, and it is what Module 04's proofs are built on.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Arora & Barak — chapter 2 (free draft)	Both definitions of NP, their equivalence, and the structural material of §4.
Cook — The Complexity of Theorem-Proving Procedures (free)	The 1971 paper , which is short and is where the question originates.
Aaronson — $P = ? NP$ (free survey)	§2 and §3 assessed honestly — the consequences, the evidence, and what the evidence is actually worth. The best thing written on the question for a general technical audience.
Ladner — On the Structure of Polynomial Time Reducibility (free)	§4's theorem , with the alternating construction.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Give certificates and verifiers for six problems**, including three not in §1's table.
- 2 **Prove the two definitions of NP equivalent**, in both directions.
- 3 **Find a problem where the certificate is not obvious** and work out what it is. Primality is a good one.
- 4 **State P versus NP precisely** and then informally, and check the informal version is faithful.
- 5 **Write the consequences of each resolution** for a domain you work in.
- 6 **Explain why $P \neq NP$ would change little operationally**.
- 7 **Assess the evidence for $P \neq NP$** and state how convincing you find it, with reasons.
- 8 **State Ladner's theorem** and explain why the constructed problem is artificial.
- 9 **Look up graph isomorphism's and factoring's current status**.
- 10 **Find a problem in your own work whose status you cannot determine**, and say what you tried.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give both definitions of NP and prove them equivalent.
- 2 Why is the certificate definition the useful one?
- 3 Give six problems with their certificates.
- 4 Which half of an NP-completeness proof does the certificate give?
- 5 State P versus NP and its informal reading.
- 6 Give four consequences of each resolution, and note the asymmetry.
- 7 Give five pieces of evidence for $P \neq NP$ and the counterweight.
- 8 State Ladner's theorem and name the natural candidates.
- 9 What does the existence of a middle mean for a problem you cannot classify?